

Staking Eth-Link API Specification

version 0.0.9

Overview

The Eth-Link API defines a provider-agnostic, minimal, and secure interface for enabling compounding ETH staking (post-Pectra). Its purpose is to allow trusted staking providers to participate in validator creation, usage reporting, and life-cycle event delivery, while the Fireblocks platform manages validator state and economic flows internally.

Eth-Link is intentionally not a full validator-management API. Instead, it provides the smallest necessary set of instructions for performing staking operations safely and consistently across multiple providers.

The platform (Fireblocks) handles:

- validator orchestration
- deposit submission
- validator and positions lifecycle tracking
- notifying providers about validators related events

Providers only implement the limited surface described in this document.

Scope & Limitations

Blockchain networks

Only ETH is supported.

Supported environments:

- **mainnet** — Ethereum production network
- **hoodi** — Ethereum testnet environment

Validator Types

Only post-Pectra compounding validators (0x02 withdrawal credentials) are supported. This API is designed exclusively for validators with compounding withdrawal credentials and does not support legacy validator types.

Fee Recipient

The fee recipient specifies the Ethereum address that receives priority fees (tips) and MEV rewards from block proposals.

Provider Responsibility: Providers must set the fee recipient internally to match the withdrawal address. This is not included in the Eth-Link API request. This ensures that all validator rewards—both consensus layer rewards (via compounding) and execution layer fees (via fee recipient)—flow to the same destination address, simplifying accounting and maintaining a clear economic model.

API Surface

The API is intentionally minimal and strict, covering only the actions required to create and operate compounding validators.

Providers

Eth-Link is intended for a small set of vetted trusted providers, each registered in the platform with:

- Provider name
- Provider ID

- Base URL for Eth-Link API (can include domain and path prefix)
- Eth-Link API keys (four keys total - see API Keys section below)

API Keys Per Environment and Network

Providers must issue **four separate API keys** to Fireblocks, organized as a matrix of environments and networks:

Environment	Network	Purpose
Production	mainnet	Production operations on Ethereum mainnet
Production	hoodi	Production operations on Hoodi testnet
Development	mainnet	Development and testing on Ethereum mainnet
Development	hoodi	Development and testing on Hoodi testnet

Why Four Keys?

Environment Separation (Prod/Dev):

- Security isolation between production and development infrastructure
- Prevents development activities from impacting production systems
- Allows independent key rotation and access control
- Meets security compliance requirements for environment segregation

Network Separation (mainnet/hoodi):

- Enables separate rate limiting policies per network
- Facilitates network-specific monitoring and alerting
- Supports independent tracing and logging per blockchain
- Allows different operational configurations (e.g., higher rate limits on testnet)

Key Naming Convention:

Providers should use a clear naming scheme for the API keys, for example:

- `fireblocks-prod-mainnet-key`
- `fireblocks-prod-hoodi-key`
- `fireblocks-dev-mainnet-key`
- `fireblocks-dev-hoodi-key`

All four keys must be securely shared with Fireblocks during the onboarding process and stored in Fireblocks' secret management system.

Base URL Examples:

The base URL can be either a simple domain or include path prefixes for API versioning or routing. It may also include network identifiers:

- `https://hoodi.provider.com`

- `https://api.provider.com/mainnet`
- `https://api.provider.com/hoodi`

The platform will append Eth-Link specific endpoint paths to this base URL. For example, if the base URL is `https://api.provider.com/mainnet` and the Eth-Link endpoint is `/eth-link/validators/create`, the full request URL will be:

```
1 https://api.provider.com/mainnet/eth-link/validators/create
```

Position Events

Add-to-stake, withdrawals and consolidation are fully orchestrated by the platform. Providers do not implement dedicated endpoints for these flows.

Validator Statuses

Beacon-chain validator statuses (activation, exit, slashing data, etc.) are obtained, if needed, in-house by the platform and are not part of this API.

Batch Validator Creation

Supported by some providers but not all. Batch creation may be considered for future versions but is currently out of scope.

Provider Responsibilities

Create Compounding Validators

Endpoint to generate validator key material and return fully signed deposit data for the platform to submit to the Ethereum deposit contract.

Health Check

A lightweight endpoint allowing the platform to verify provider readiness and availability. Polled approximately every 60 seconds.

Event Callbacks

Webhook-style event delivery for key validator lifecycle events, including:

- cancellation
- deposit

Response Authentication

The Create Validator endpoint must authenticate responses using HMAC-SHA256 to ensure authenticity and prevent tampering of sensitive validator data. The Health Check endpoint does not require HMAC authentication as it returns only non-sensitive status information.

HMAC Signature for Create Validator

Providers must include an `x-signature` header in Create Validator responses. The signature is calculated as:

```
1 HMAC-SHA256(requestId:timestamp:depositDataRoot, apiKey)
```

Where:

- `requestId` is the value from the `x-request-id` request header (provided by Fireblocks)
- `timestamp` is the Unix timestamp (seconds since epoch) **generated by the provider** when creating the response
- `depositDataRoot` is the SSZ Merkle root from the deposit data being returned (generated by the provider)

- `:` is the colon delimiter separating the components
- `apiKey` is the shared secret API key

Important: The provider generates the timestamp, not Fireblocks. The provider includes this timestamp in both the HMAC calculation and the `x-timestamp` response header. This allows Fireblocks to verify the signature using the timestamp from the response header.

Note on Consistency: This HMAC pattern (including timestamp) is consistent with Fireblocks' Network Link API, enabling providers to reuse HMAC generation logic across multiple Fireblocks integrations.

The provider must also include the `x-timestamp` header in the response with the same timestamp value used in the signature calculation.

Example (TypeScript/Node.js):

```

1 import crypto from 'crypto';
2
3 function generateHMACSignature(
4   requestId: string,
5   timestamp: string,
6   depositDataRoot: string,
7   apiKey: string
8 ): string {
9   const stringToSign = `${requestId}:${timestamp}:${depositDataRoot}`;
10  const hmac = crypto.createHmac('sha256', apiKey);
11  hmac.update(stringToSign);
12  return hmac.digest('hex');
13 }
14
15 // Usage
16 const requestId = req.headers['x-request-id']; // From Fireblocks
17 // request
18 const timestamp = Math.floor(Date.now() / 1000).toString(); //
19 // Provider generates this
20 const depositDataRoot =
21   '0xfd997ecea9711498990181f0e3f8426b1c9528e61a8fbcc999db5fb3b3d6ca4b';
22 const apiKey = 'your-secret-api-key'; // From your configuration
23
24 const signature = generateHMACSignature(requestId, timestamp,
25   depositDataRoot, apiKey);
26
27 // Response headers
28 headers['x-request-id'] = requestId; // Echo from request
29 headers['x-timestamp'] = timestamp; // Your generated timestamp
30 headers['x-signature'] = signature; // Your calculated HMAC

```

HMAC Verification (Platform Side)

The platform verifies the signature by:

1. Extracting `x-request-id`, `x-timestamp`, `x-signature` from response headers and `depositDataRoot` from response body
2. Reconstructing the string: `requestId:timestamp:depositDataRoot`
3. Calculating HMAC-SHA256 using the same API key
4. Comparing the calculated signature with the received signature using a timing-safe comparison
5. Validating that the timestamp is recent (within acceptable time window)

Providers must keep their API keys secure and rotate them when needed through the manual coordination process described in the Authentication section.

Validator Housekeeping and Retention

Providers must maintain their own internal records and housekeeping for provisioned validators. While the platform sends lifecycle events (such as deposit confirmations and cancellations), providers should not rely 100% on these events as their sole source of truth for validator state management.

Validator retention periods, key management obligations, liability for slashing, and protection of customer funds are subject to the legal agreement between the provider and Fireblocks and are out of scope of this API specification.

Linear Provisioning Flow

The platform follows a **strictly linear provisioning flow**:

- Validators are created only upon explicit client request
- The platform does not pre-provision, pool, or hold validators ahead of time
- Each validator creation request is tied to a specific user action

This means providers can expect validator creation requests to be made on-demand rather than in batches or advance allocations.

Conventions

This section defines the common data conventions used across all Eth-Link API endpoints. These rules ensure consistency between providers, eliminate ambiguity, and simplify client integrations.

Identifiers

Eth-Link uses **UUID v4 strings** as unique identifiers for requests, resources, and correlation purposes.

Example:

"3d8e7c9d-4b61-4a54-9ec0-2e9dd5b7c3af"

Timestamps

All timestamps in the API use **ISO 8601 format** with UTC timezone.

Format: YYYY-MM-DDTHH:mm:ss.sssZ

Examples:

- "2025-01-22T14:30:00Z"
- "2025-01-22T14:30:00.123Z" (with milliseconds)

Unix Timestamps: For HMAC signature generation, Unix timestamps (seconds since epoch) are used. These are represented as strings containing the number of seconds. The provider generates these timestamps when creating responses.

Blockchain Network

Eth-Link supports two Ethereum blockchain networks:

Blockchain	Description
mainnet	Ethereum production network
hoodi	Ethereum testnet used for pre-production validation

The **blockchain** field must always contain one of these exact string literals.

Monetary Amounts

Monetary amounts in Eth-Link API requests are represented as **strings in ETH units**.

Examples:

- "32" (standard single validator)
- "42.5" (non-standard amount)
- "1024" (large deposit)

Note: This is conceptually different from the `amount` field in response `depositData` structures, which is a numeric integer in gwei as required by the Ethereum deposit contract protocol specification.

Casing Convention

Eth-Link uses **camelCase** for all field names.

Example:

Instead of:

- `withdrawal_credentials`
- `withdrawal-credentials`

Eth-Link uses:

- `withdrawalCredentials`

HTTP Headers

All HTTP headers use **lowercase** naming convention.

Example:

Instead of:

- `X-Request-ID`
- `X-API-Key`

Eth-Link uses:

- `x-request-id`
- `x-api-key`

Request ID

Every API request must include a unique request ID in the HTTP header `x-request-id` (UUID format). This ensures idempotency and enables safe retries. Providers must return the same request ID in the response header `x-request-id` for traceability.

Providers are responsible for implementing proper idempotency handling:

- **Same request, same payload:** Respond consistently with the same result
- **Same request, different payload:** Return an appropriate error (e.g., 409 Conflict)
- **Retention:** Retain request ID values for at least 24 hours to detect duplicates

Example:

```

1 Request:
2 x-request-id: 550e8400-e29b-41d4-a716-446655440000
3 x-api-key: <provider_api_key>
4
5 Response (Create Validator):
6 x-request-id: 550e8400-e29b-41d4-a716-446655440000
7 x-timestamp: 1706014800
8 x-signature:
9   a52454175a8516b7b2176379e06a9d7d5fa0702c38fc45e3ef63bf1a5746c0c0

```

```
10 Response (Health Check):
11 x-request-id: 550e8400-e29b-41d4-a716-446655440000
```

Client ID

The Create Validator endpoint includes a client ID in the HTTP header `x-client-id` to identify the end client initiating the validator creation. The client ID is consistently derived from Fireblocks internal tenant ID but does not directly translate to it, ensuring privacy while maintaining traceability. Providers must treat this as an opaque string and must not validate or assume any specific format — it may resemble a UUID but is not guaranteed to be one.

The `x-client-id` header enables providers to segment metrics and logs by end client, track validator creation patterns per client, support multi-tenant analytics and reporting, and maintain audit trails for compliance purposes.

Note: The client ID is not included in other endpoints (such as Send Validator Event or Health Check) as it can be derived from the validator's public key, which was associated with the client during validator creation.

Example:

```
1 Request:
2 x-request-id: 550e8400-e29b-41d4-a716-446655440000
3 x-client-id: 7f3e9a2b-4c1d-4e5f-9a8b-3c2d1e0f4a5b
4 x-api-key: <provider_api_key>
```

Authentication

All Eth-Link requests sent from the platform (Fireblocks) to the provider must be authenticated.

Eth-Link uses a simple, robust API key mechanism based on a shared secret issued by the provider.

API Key Header

Every authenticated request includes an HTTP header:

```
1 x-api-key: <api-key>
```

- The header name uses lowercase convention.
- The API key is a provider-generated secret string.
- The platform includes this header on **all Eth-Link API calls**, except where explicitly documented otherwise.

Provider Responsibilities

Providers **must**:

1. Validate the API key

Each request must check the presence and validity of the `x-api-key` header.

If missing or invalid:

- Return **401 Unauthorized**
- Include a structured error response:

```
1 {
2   "message": "Unauthorized",
3   "code": "UNAUTHORIZED"
4 }
```

2. Keep the key secure

Providers must store the API key securely and treat it as a sensitive secret.

3. Support key rotation

Providers should offer a mechanism to:

- Rotate the API key without downtime

- Allow temporary overlap where old and new keys are both accepted
- Document rotation procedures to the platform if needed
- Providers can notify the platform about key rotation using health-check endpoint response.

Platform Responsibilities

The platform (Fireblocks) ensures:

- The correct API key is attached to every request
- No requests are sent without an authentication header unless explicitly allowed by the provider

IP Whitelisting

All API calls from Fireblocks to the provider service are sent from a fixed set of IP addresses, grouped by geographical region. Providers should whitelist these addresses to allow Fireblocks to access the provider servers.

Providers must configure their firewalls, security groups, or network access control lists to allow inbound HTTPS traffic from these IP addresses. Whitelisting these IPs adds an additional layer of security by ensuring that only requests originating from Fireblocks infrastructure can reach the provider's Eth-Link API endpoints.

IP Address List: For the complete list of Fireblocks IP addresses by region, please refer to the [Network Link Integration Guide - IP Allowlisting](#) section.

Note: If Fireblocks needs to add or change IP addresses, this will be communicated to providers with advance notice through the standard coordination process.

Errors

Eth-Link defines a consistent error model across all endpoints.

Returning correct and meaningful error responses is the responsibility of the provider.

Providers must ensure that all failures—validation errors, malformed inputs, missing fields, processing errors, or unexpected conditions—are reported using the structure and status codes defined in this section.

Error Response Structure

All error responses MUST follow this structure:

Field	Type	Description
message	string	Human-readable error message explaining what went wrong.
code	string	Machine-readable error code for programmatic handling.

Example:

```
1 {  
2   "message": "Invalid validator public key format",  
3   "code": "INVALID_ARGUMENT"  
4 }
```

HTTP Status Codes

Providers should use standard HTTP status codes and pair them with appropriate error codes:

HTTP Status	Recommended Error Code	When to Use
400 Bad Request	<code>INVALID_REQUEST</code> , <code>INVALID_ARGUMENT</code>	Missing required fields, malformed JSON, invalid field formats, wrong enum values
401 Unauthorized	<code>UNAUTHORIZED</code>	Missing or invalid <code>x-api-key</code> header
404 Not Found	<code>NOT_FOUND</code>	Unknown validator pubkey or resource
409 Conflict	<code>CONFLICT</code>	Duplicate request ID with different payload, invalid state transition
429 Too Many Requests	<code>RATE_LIMITED</code>	Request rate exceeds provider limits
500 Internal Server Error	<code>INTERNAL_ERROR</code> , <code>DEPENDENCY_FAILURE</code>	Unexpected failures, upstream service issues
503 Service Unavailable	<code>SERVICE_UNAVAILABLE</code>	Temporary provider unavailability, maintenance

Error Codes

Code	Description	Common Scenarios
UNAUTHORIZED	Authentication failed	Missing or invalid API key
INVALID_REQUEST	Request structure is invalid	Malformed JSON, missing required fields, incorrect request format
INVALID_ARGUMENT	Field value is invalid	Invalid pubkey format, bad Ethereum address, unsupported enum value, invalid timestamp format
NOT_FOUND	Resource not found	Unknown validator pubkey
CONFLICT	Request conflicts with current state	Duplicate request ID with different payload,

		invalid validator state transition
RATE_LIMITED	Too many requests	Provider throttling to protect systems
DEPENDENCY_FAILURE	Upstream dependency unavailable	Signing service down, database unavailable, key management system unreachable
INTERNAL_ERROR	Unexpected provider error	Unhandled exceptions, unexpected failures
SERVICE_UNAVAILABLE	Provider temporarily unavailable	Maintenance mode, system overload

Note: This is a minimal error code set for v1. Future versions may introduce more specific error codes for finer-grained error handling based on operational needs.

Endpoint - Create New Validator

POST `{baseUrl}/eth-link/validators/create`

Creates a new compounding ETH validator and returns fully signed deposit data that can be submitted to the Ethereum Deposit Contract.

This endpoint is used when spawning a new validator under the provider's management.

The `{baseUrl}` is taken from the provider's configuration in the platform. The platform appends the endpoint path to construct the full request URL.

Request Headers

Header	Type	Required	Description
x-request-id	string (UUID)	✓	Unique identifier for this request. Used for idempotency, correlation, and troubleshooting across systems.
x-client-id	string	✓	Identifier for the end client initiating this validator creation. Used for tracking, analytics, and audit purposes.

x-api-key	string	✓	Provider-issued API key for authentication.
------------------	--------	---	---

Request Body

```

1 {
2   "blockchain": "hoodi",
3   "withdrawalAddress": "0xE40F80618324C814cD444434670a44ba4583aE38",
4   "region": "eu",
5   "amount": "32"
6 }
```

Request Field Definitions

Field	Type	Required	Description
blockchain	string	✓	Target execution blockchain network - <code>mainnet</code> or <code>hoodi</code> .
withdrawalAddress	string	✓	Ethereum execution-layer address that will receive funds upon validator exit. Must be a valid <code>0x</code> - prefixed EVM address.
region	string		Optional. Provider hosting region for the validator. Valid values: <code>US</code> or <code>EU</code> . Used for physical or jurisdictional placement of validator nodes.
amount	string	✓	Amount in ETH (not gwei) for the initial deposit. Examples: <code>"32"</code> , <code>"42.5"</code> , <code>"1024"</code> .

Note: The `feeRecipient` field is not included in the request. Providers must set the fee recipient internally to match the withdrawal address. This ensures that all validator rewards—both consensus layer rewards (via compounding) and execution layer fees (via fee recipient)—flow to the same destination address.

Response Headers

Header	Type	Required	Description
x-request-id	string (UUID)	✓	Echo of the request ID from the request header for traceability.
x-timestamp	string	✓	Unix timestamp (seconds since epoch) when the response was generated by the provider. Used in HMAC signature calculation.
x-signature	string (hex)	✓	HMAC-SHA256 signature authenticating the response. Calculated as: HMAC - SHA256(req uestId:tim estamp:dep ositDataRo ot, apiKey)

Response Body

```
1 {
2   "data": {
3     "depositData": {
4       "pubkey":
5       "0x8fc7b098767d6b45f431ac820f6884f6f74bbebee4e6f30456bbcf48a280248307a
6       8260f2913a27c076f2f0c64c67aad",
7       "withdrawalCredentials":
8       "0200000000000000000000000000e1f4acc0affb36a805474e3b6ab786738c6900a2",
9       "depositDataRoot":
10      "0xfd997ecea9711498990181f0e3f8426b1c9528e61a8fbcc999db5fb3b3d6ca4b",
11      "signature":
12      "0x802bec470e333c380a0c530361e76cfa27f662a411d993a663aa080070601411aae
13      1263ccbd8f812199b1fff091e20fc0b7e793c1d0313d36d27040588d578fd8b6029be2
14      64cf79533132d1df0d40dc9ce5ba159fe4c936e9c4b4c9bf2988a29",
15      "amount": 3200000000
16    }
17  }
```

Response Field Definitions

Top-level fields

Field	Type	Description
data	object of type NewValidatorsResponse	

NewValidatorsResponse fields

Field	Type	Description
depositData	DepositData	Container for all deposit-message values required by the Ethereum deposit contract.

DepositData fields

These fields correspond to the standard Ethereum *DepositData* structure as defined in the consensus-layer specs (post-Pectra).

Field	Type	Description
pubkey	string (48-byte hex)	BLS public key of the validator. This becomes the validator identity used in consensus duties.
withdrawalCredentials	string (32-byte hex)	Withdrawal credentials formatted according to Pectra notation with 0x02000000000000000000000000000000 . Determines where funds will be sent after exit.
depositDataRoot	string (32-byte hex)	SSZ Merkle root of the entire deposit message. Must match the root calculated by the validator client. Used by the Deposit Contract to validate the deposit. This value is also included in the HMAC signature for

		response authentication.
signature	string (96-byte hex)	BLS12-381 signature over the deposit message. Required by the Ethereum Deposit Contract.
amount	number (gwei)	Deposit amount in gwei as required by the Ethereum deposit contract specification. This is the request amount converted to gwei (multiply by 1,000,000,000).

Behavior and Guarantees

- The provider **generates** the validator keypair and produces the full **DepositData** structure.
- The response contains all fields needed for calling the **Ethereum Deposit Contract**.
- The caller is responsible for **submitting** the deposit transaction to Ethereum.
- The HMAC signature in the **x-signature** header ensures that the response is authentic and came from the provider, and that the deposit data has not been tampered with.
- The **depositDataRoot** in the HMAC provides cryptographic integrity for all deposit data fields.
- The **x-request-id** header ensures end-to-end traceability and safe retries.
- The **x-client-id** header allows providers to track validator creation per end client for analytics and audit purposes.
- Providers must set the validator's fee recipient to match the withdrawal address internally.
- Providers must convert the request **amount** (in ETH) to gwei for the deposit data structure.
- **Timeout SLO:** The platform applies a **30-second timeout** to this endpoint. Providers are expected to respond within this window under normal operating conditions. If a provider cannot respond in time, the platform will treat the request as failed and may retry using the same **x-request-id** for idempotency.

Send Validator Event

POST `{baseUrl}/eth-link/validators/{pubkey}/events`

Sends a validator lifecycle event from the platform (Fireblocks) to the provider.

Used to notify the provider about meaningful validator state transitions such as creation, deposit submission, activation, exit, cancellation, or withdrawal completion.

The **{baseUrl}** is taken from the provider's configuration in the platform. The platform appends the endpoint path to construct the full request URL.

Requests are authenticated using the provider-issued API key (**x-api-key**).

Request

Path Parameter

Field	Type	Description
pubkey	string (hex)	Validator BLS public key. 48 bytes, hex-encoded, prefixed with 0x .

Request Headers

Header	Type	Required	Description
x-request-id	string (UUID)	✓	Unique identifier for the request. Used for idempotency and tracing.
x-api-key	string	✓	Provider-issued API key for authentication.

Request Body

```
1 {
2   "eventType": "DEPOSIT_SUBMITTED",
3   "timestamp": "2025-01-22T14:30:00Z",
4   "details": {
5     "txHash": "0x4b5d...e80",
6     "amount": "32"
7   }
8 }
```

Field	Type	Description
eventType	string	Lifecycle event type.
timestamp	string (ISO 8601)	ISO 8601 timestamp (UTC) when the event occurred. Example: "2025-01-22T14:30:00Z"
details	object	Event-specific metadata. Structure depends on eventType (e.g., tx hash, epochs, reason codes)

Event Types

DEPOSIT_SUBMITTED

Indicates that the platform has submitted the validator's deposit transaction to the Ethereum deposit contract.

details object:

Field	Type	Description
txHash	string (hex)	Transaction hash of the deposit.
amount	string	Amount deposited, in ETH units.

Example:

```
1 "details": {
2   "txHash": "0x4b5d...e80",
3   "amount": "32"
4 }
```

VALIDATOR_CANCELED

Indicates that the platform will not proceed with this validator, and the provider may safely dispose of the key material and any pre-provisioned resources.
Valid only before a deposit has been submitted.

details object:

May be empty or may include optional diagnostic information.

Example:

```
1 "details": {
2   "reason": "USER_REQUEST"
3 }
```

Response Headers

Header	Type	Description
x-request-id	string (UUID)	Echo of the request ID from the request header for traceability.

Success (200 OK)

```
1 {}
```

The response body may be empty or contain an empty JSON object. The success is indicated by the 200 status code and the echoed `x-request-id` header.

Endpoint - Health Check

GET `{baseUrl}/eth-link/health`

Allows the platform (Fireblocks) to verify that a provider's Eth-Link implementation is available, responsive, and ready to serve requests.

The `{baseUrl}` is taken from the provider's configuration in the platform. The platform appends the endpoint path to construct the full request URL.

This endpoint is typically polled on a recurring basis (approximately every 60 seconds) to:

- decide whether a provider is currently eligible for routing user operations, and
- surface provider availability in internal dashboards and logs.

Authentication using the provider-issued API key (`x-api-key`) is required.

Note: This endpoint does not require HMAC response authentication as it returns only non-sensitive status information. The high polling frequency (every 60 seconds) also makes HMAC signatures unnecessary overhead.

Request Headers

Header	Type	Required	Description
x-request-id	string (UUID)	✓	Unique identifier for the request. Used for tracing and correlation.
x-api-key	string	✓	Provider-issued API key for authentication.

No request body is required for this endpoint.

Response

Response Headers

Header	Type	Description
x-request-id	string (UUID)	Echo of the request ID from the request header for traceability.

Success (200 OK)

```
1 {
2   "healthy": true,
3   "timestamp": "2025-01-22T14:30:00Z",
4   "message": "Operating normally"
5 }
```

Field	Type	Description
healthy	boolean	Indicates whether the provider is ready to accept Eth-Link requests. <code>true</code> = ready to handle all operations. <code>false</code> = unable to reliably process requests.
timestamp	string (ISO 8601)	ISO 8601 timestamp (UTC) when the health status was generated.
message	string	Optional. Any additional information or context from the provider (e.g.,

		warnings, maintenance notices, or operational notes).
--	--	---

Field **healthy** Values:

Value	Meaning
true	Provider is fully operational and ready to accept Eth-Link requests. The platform will route validator creation and other operations to this provider.
false	Provider cannot reliably handle requests at this time. The platform will not route new operations to this provider until it reports healthy again.